5.派生类的应用讨论

赋值兼容:

公用派生类完整地继承了基类的功能，可将公有派生类的值赋给基类对象，在用到基类对象时可用其派生类对象代替。

5.派生类的应用讨论

赋值兼容具体表现在以下几个方面:

(1) 派生类对象可向基类对象赋值。

(2) 派生类对象可替代基类对象向基类对象的引用进行赋值或初始化。

(3) 若函数的参数是基类对象或基类对象的引用，其实参可以是派生类对象。

(4) 派生类对象的地址可以赋给指向基类对象的指针变量。

信息科学与工程学院
王文博 2026

5.派生类的应用讨论

(1)派生类对象可向基类对象赋值

可用公用派生类对象对其基类对象赋值。

如：

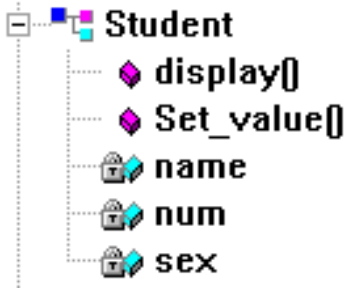
```
Student a1;
Student1 b1;
a1=b1;
```

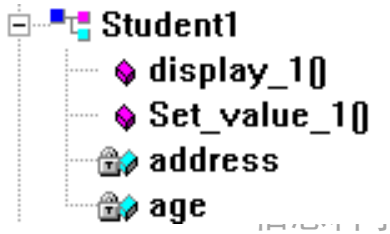
定义基类Student对象a1

定义类Student的公有派生类Student1的对象b1

用派生类Student1对象b1对基类对象a1赋值

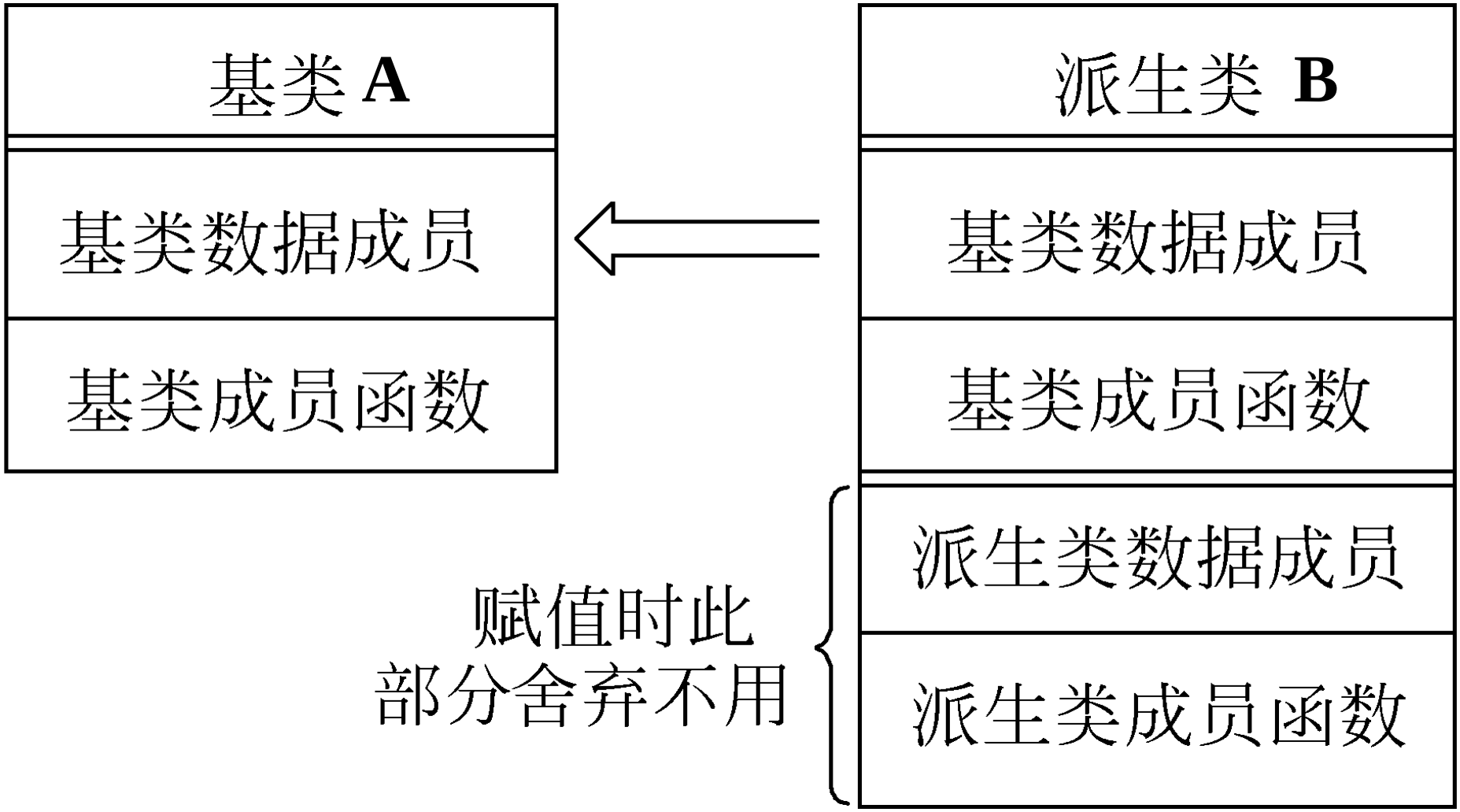
在赋值时舍弃派生类自己的成员。





5.派生类的应用讨论

(1)派生类对象可向基类对象赋值



信息科学与工程学院
王文博 2026

5.派生类的应用讨论

(1)派生类对象可向基类对象赋值

该赋值兼容关系是单向的、不可逆的：

基类对象不包含派生类的成员，无法对派生类的成员赋值。

同一基类的不同派生类对象之间也不能赋值。

如：

```
a1.age=23;      //错误，a1中不包含派生类中增加的成员

b1.age=21;      //正确，b1中包含派生类中增加的成员
```


王文博 2026

5.派生类的应用讨论

(2)派生类对象可替代基类对象，向基类对象的引用进行赋值或初始化

可定义A对象的引用变量:

```
B b1;                  //定义公用派生类B对象b1
A& r=b1;
```

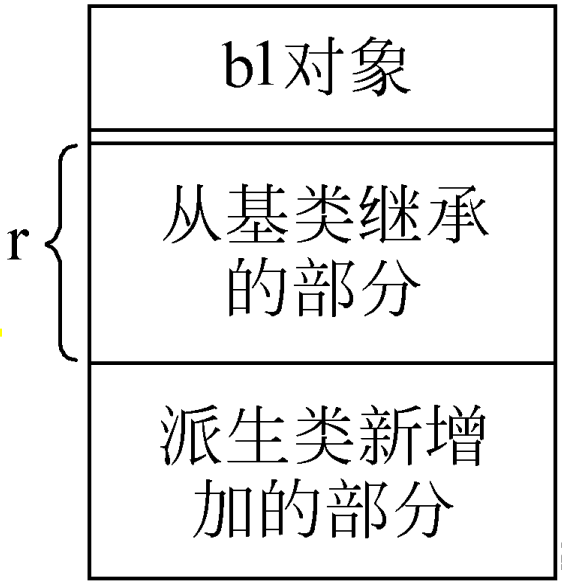
定义基类A对象的引用变量r，并用派生类B对象b1对其初始化

5.派生类的应用讨论

(2)派生类对象可替代基类对象，向基类对象的引用进行赋值或初始化

注意:

- **r不是b1的别名。**
- **r是b1中基类部分的别名，r与b1中基类部分共享同一段存储单元。**
- **r与b1有相同的起始地址。**



5.派生类的应用讨论

(2)派生类对象可替代基类对象，向基类对象的引用进行赋值或初始化

也可以保留A& r=a1;，对r重新赋值：

```
A a1;          //定义基类A对象a1
B b1;          //定义公用派生类B对象b1
A& r=a1;
r=b1;
```

用派生类B对象b1中的基类成员的值对r重新赋值

5.派生类的应用讨论

(3) 若函数的参数是基类对象或基类对象的引用，其实参可以是派生类对象。

如有一函数：

```
void fun(A& r)      //形参是类A的对象的引用变量
{   cout<<r.num<<endl;   }
```

调用fun函数时可用派生类B的对象b1作实参：

```
fun(b1);
```

➡ 输出类B的对象b1的基类数据成员num的值。

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

基类

```
class Person{
        string IdPerson;
        string Name;
        Tsex Sex;
        int Birthday;
        string HomeAddress;
public:
        Person(string,string,Tsex,int,string);
        Person();
        Person(Person &ps);
        ~Person();
        Person & operator=(Person &);
        void PrintPersonInfo();
};
```

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

基类的复制构造函数

```
Person::Person(Person &ps)
{
        IdPerson=ps.IdPerson;
        Name=ps.Name;
        Sex=ps.Sex;
        Birthday=ps.Birthday;
        HomeAddress=ps.HomeAddress;
}
```

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

```
Person & Person::operator=(Person &ps)
{
        IdPerson=ps.IdPerson;
        Name=ps.Name;
        Sex=ps.Sex;
        Birthday=ps.Birthday;
        HomeAddress=ps.HomeAddress;
        return *this;
}
```

基类的赋值复制操作符重载

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

```
class Student:public Person
{
        string NoStudent;     //学号
        course cs[30];        //30门课程与成绩
public:
        Student(string id,string name,Tsex sex,int birthday,string
homeadd,string nostud);
        Student();
        Student(Student &Std);
        ~Student();
        Student & operator=(Student &Std);
        int SetCourse(string,int);
        int GetCourse(string);
        void PrintStudentInfo();
};
```

公有派生类

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

```
Student::Student(Student &Std):Person(Std)
{
        int i;
        NoStudent=Std.NoStudent;
        for(i=0;i<30;i++){
          cs[i].coursename=Std.cs[i].coursename;
          cs[i].grade=Std.cs[i].grade;
        }
}
```

按赋值兼容规则Std可为Person实参

派生类的复制构造函数

5.派生类的应用讨论

赋值兼容规则与自定义复制构造函数

```
Student & Student::operator=(Student &Std)
{
        int i;
        Person::operator=(Std);
        NoStudent=Std.NoStudent;
        for(i=0;i<30;i++){
        cs[i].coursename=Std.cs[i].coursename;
        cs[i].grade=Std.cs[i].grade;
        }
        return *this;
}
```

调用基类的复制赋值操作符

完整代码示例

5.派生类的应用讨论

(4) 指向基类对象的指针变量也可指向派生类对象。

```
Student *a1;      //定义基类Student指针a1
Student1 b1;      //公有派生类Student1对象b1
a1=&b1;
```

用指向基类对象的指针变量指向其派生类对象是合法的、安全的，不会出现编译上的错误。

但是：只能通过该指针访问派生类中由基类继承来的成员。

5.派生类的应用讨论

(4) 指向基类对象的指针变量也可指向派生类对象。

```
#include <iostream>
#include <string>
using namespace std;
class Student
{
public:
   Student(int, string, float);
   void display( );
private:
   int num;
   string name;
   float score;
};
```

5.派生类的应用讨论

(4) 指向基类对象的指针变量也可指向派生类对象。

```
Student::Student(int n, string nam,float s)
{    num=n;    name=nam;    score=s;     }

void Student::display( )
{
      cout<<endl<<"num:"<<num<<endl;
      cout<<"name:"<<name<<endl;
      cout<<"score:"<<score<<endl;
}
```

5.派生类的应用讨论

```
class Graduate:public Student
{
public:
    Graduate(int n, string nam,float s,float p):
    Student(n,nam,s) {  pay=p;  }
    void display( )
    {
       Student::display();
       cout<<"pay="<<pay<<endl;
     }
private:
    float pay;    //工资
};
```

5.派生类的应用讨论

```
void main( )
{
  Student stud1(1001,"Li",87.5);
  Graduate grad1(2001,"Wang",98.5,563.5);
  Student *pt=&stud1;
  pt->display( );
  pt=&grad1;
  pt->display( );
}
```

定义指向Student类对象的指针,并指向stud1

调用stud1.display函数

指针指向派生类对象grad1

调用？？

5.派生类的应用讨论

```
void Student::display( )
{
    cout<<endl<<"num:"<<num<<endl;
    cout<<"name:"<<name<<endl;
    cout<<"score:"<<score<<endl;
}
```

96

```
void display( )
{
    Student::display();
    cout<<"pay="<<pay<<endl;
}
```

注意，调用的是派生类继承的基类成员，派生类的输出无效

输出：

num:1001
name:Li
score:87.5

num:2001
name:wang
score:98.5

继承与组合（聚合）

定义：在一个类中以另一个类的对象作为数据成员的，称为类的组合（聚合）。

例：

```
//教师类
class Teacher
{public:
  ┆
private:
   int num;
   string name;
   char sex;
 };
```

```
//生日类
class BirthDate
{ public:
   ┆
private:
int year;
int month;
int day;
};
```

```
//教授类
class Professor:public
                Teacher
{  public:
      ┆
  private:
  BirthDate birthday;
  //BirthDa        对象作为数据成员
};
```

继承

聚合：BirthDate *birthday;

继承与组合（聚合）

Professor类通过继承，从Teacher类得到了num, name, age, sex等数据成员，通过组合，从BirthDate类得到了year, month, day等数据成员。

继承是纵向的，组合是横向的

- **任一基类在派生类中只能继承一次，否则会引起成员名的冲突；**
- **若在派生类中，确实要有二个以上基类的成员，则可用基类的二个对象作为派生类的成员。**

信息科学与工程学院
王文博 2026

继承与组合（聚合）

把一个类作为派生类的基类或把一个类的对象作为一个类的成员，在使用上是有区别的：

- 在派生类中可直接使用基类的成员（访问权限允许的话）
- 使用对象成员的成员时，必须在对象名后加上成员运算符“.”和成员名。

所有成员都可以访问吗？

派生类与模板

- 共同点：代码的复用和程序的通用性。
- 区别：
  - 类模板是相互独立的，未使用继承的思想，模板追求的是运行效率。
  - 派生类一般由简到繁，逐步深入，一步步完成一个复杂的任务。追求的是编程的效率。

6.多态性与虚函数

什么是多态性？

一个接口，多种方法：具有不同功能的函数可以用同一个函数名，这样就可以用一个函数名调用不同内容的函数。

编译时的多态性 ← 函数重载

运算符重载

运行时的多态性

程序执行前，无法确定调用哪一个函数，需在执行过程中动态确定

6.多态与虚函数

在面向对象方法中一般是这样表述多态性的：向不同的对象发送同一个消息，不同的对象在接收时会产生不同的行为（即方法）。也就是说，每个对象可以用自己的方式去响应共同的消息。

从简单的实例出发→

6.多态性与虚函数

多态简单示例

```
#include <iostream>
using namespace std;
class CShape
{
public:
        void  display( ) {cout<< "Shape\n";}
};
class CEllipse: public CShape
{
public:
        void display( ) {cout<< "Ellipse\n";}
};
class CCircle : public CShape
{
public:
        void display( ) {cout << "Circle\n";}
};
```

```
int main()
{
        CShape Shape;
        CEllipse Ellipse;
        CCircle Circle;
        CShape* pShapeArray[3] =
                        {&Shape, &Ellipse, &Circle};
        for(int i=0;i<3;i++)
                pShapeArray[i]->display();
        return 0;
}
```

输出：
Shape
Shape
Shape

希望输出内容

输出：
Shape
Ellipse
Circle

多态简单示例

学院
2026

多态简单示例

```
#include <iostream>
using namespace std;
class CShape
{
public:
        void virtual display( ) {cout<< "Shape\n";}
};
class CEllipse: public CShape
{
public:
        void display( ) {cout<< "Ellipse\n";}
};
class CCircle : public Cshape
 {
public:
        void display( ) {cout << "Circle\n";}
};
```

多态简单示例

```
int main()
{
        CShape Shape;
        CEllipse Ellipse;
        CCircle Circle;
        CShape* pShapeArray[3] =
                        {&Shape, &Ellipse, &Circle};
       for(int i=0;i<3;i++)
              pShapeArray[i]->display();
}
```

输出：
Shape
Ellipse
Circle

虚函数的作用

6.1 虚函数

定义：

虚函数是一个类的成员函数，声明的格式：

virtual　返回类型　函数名（参数表）；

virtual仅用于类内定义中，若虚函数在类外定义，不能加virtual。

虚函数必须是类的一个成员函数，不能是友元函数，也不能是静态的成员函数

当某一个类的成员函数被定义为虚函数，则由该类派生的所有派生类中，该函数始终保持虚函数特征。

信息科学与工程学院
王文博 2026

6.1 虚函数

虚函数的作用

- 允许在派生类中重新定义与基类同名的函数。
- 可以通过基类指针或引用来访问派生类中和基类同名的函数。
- 实现运行时的多态性

多态简单示例1_1_V

6.1 虚函数

虚函数的使用方法

(1)在基类用virtual声明成员函数为虚函数。

(2) 在派生类中重新定义此函数:注意函数名、函数类型、参数个数及其类型必须相同。

(3) 定义一个指向基类对象的指针变量或引用，并使它指向同一类族中需要调用该函数的对象。

(4) 通过该指针变量或引用调用此虚函数。

6.1 虚函数

在什么情况下应当声明虚函数？

- 成员函数所在的类作为基类
- 该基类被继承后，其成员函数被更改功能
- 派生类访问方式：基类指针或引用

虚函数与一般的成员函数相比较，调用时的执行速度要慢一些。为了实现多态性，在每一个派生类中均要保存相应虚函数的入口地址表，函数的调用机制也是间接实现的。

6.1 虚函数

虚函数的调用示例2

基类声明

```
class Student
{
        string coursename;
        int classhour;
        int credit;
 public:
        Student()
        {       coursename="#";classhour=0;credit=0;}
        virtual void Calculate(){credit=classhour/16;}
        void SetCourse(string str,int hour)
        { coursename=str;    classhour=hour;       }
        int GetHour(){return classhour;}
        void SetCredit(int cred){credit=cred;}
        void Print()
        {cout<<coursename<<'\t'<<classhour<<"学时
        "<<'\t'<<credit<<"学分"<<endl;}
};
```

虚函数

信息科学与工程学院
王文博 2026

6.1 虚函数

虚函数的调用示例2

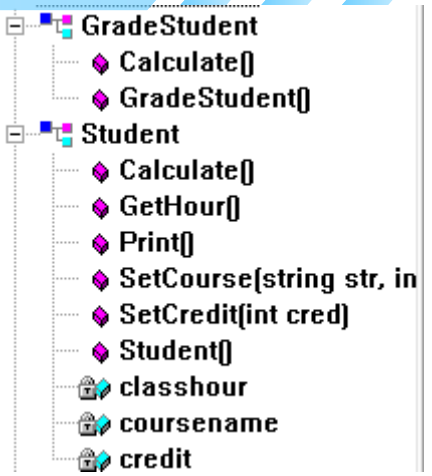
公有派生类声明

```
class GradeStudent:public Student
{
  public:
      GradeStudent(){ };
      void Calculate(){SetCredit(GetHour()/20);}
};
```

虚函数

6.1 虚函数

虚函数的调用示例2




```
Student s,*ps;
GradeStudent g;
s.SetCourse(“物理”,80);
s.Calculate();
g.SetCourse(“物理”,80);
 g.Calculate();
cout<<"本科生："<<'\t';        s.Print();
cout<<"研究生："<<'\t';        g.Print();
s.SetCourse("数学",160);
g.SetCourse("数学",160);
ps=&s;
ps->Calculate();
cout<<"本科生："<<'\t';
ps->Print();
ps=&g;
ps->Calculate();
cout<<"研究生："<<'\t';
ps->Print();
return 0;
}
```

调用**student**成员函数

Student指针指向**student**类对象

调用派生类新定义的成员函数

student指针指向派生类**Gradestudent**类对象

通过基类指针调用虚函数实现多态

6.1 虚函数

虚函数的调用示例2

```
void Calfun(Student &ps,string str,int hour
{
        ps.SetCourse(str,hour);
        ps.Calculate();
        ps.Print();
}
```

通过引用调用虚函数实现多态

```
int main(){
        Student s;
        GradeStudent g;
        cout<<"本科生:";
        Calfun(s,"物理",80);
        cout<<"研究生:";
        Calfun(g,"物理",80);
        return 0;
}
```

派生类对象作为基类引用的实参,只有**calculate()**为虚函数才能实现动态的多态性

6.1 虚函数

用new运算符建立临时对象，且用指向基类的指针指向该临时对象。

如：Student *p=new GradeStudent;

delete p;

问题：根据赋值兼容规则系统只执行基类的析构函数，而不执行派生类的析构函数。

```
#include<iostream>
using namespace std;
class A
{
public:
            ~A( )
{  cout<<"A析构\n";   }
};
class B:public A
{
public:
      B( );
      ~B( );
private:
      int *p;
};
```

```
B::B( ){ p=new int( );  }
B::~B( )
{
     cout<<"B析构\n";
     delete p;
}
int main( )
{
    A *a=new B;
    delete a;
}
```

输出：A析构

通过基类A指针撤销一个派生类B对象时，如果~A非virtual，仅调用A类中的析构函数，这里会造成类B里p指针指向的内存出现泄露

虚析构函数

解决办法：

把析构函数定义为虚函数，实现撤消对象时的多态性。

虚析构函数声明： virtual ~类名( );

何时需要虚析构函数？

当通过基类指针删除派生类对象时，当派生类对象有重要的析构函数时（如撤销动态内存分配），需要调用派生类的析构函数时，就需要让基类的析构函数成为虚函数。

信息科学与工程学院
王文博 2026

```
#include<iostream>
using namespace std;
class A
{
public:
   virtual ~A( )
{  cout<<"A析构\n";   }
};
class B:public A
{
public:
       B( );
       ~B( );
private:
       int *p;
};
```

```
B::B( ){ p=new int( );  }
B::~B( )
{
      cout<<"B析构\n";
      delete p;
}
int main( )
{
     A *a=new B;
     delete a;
}
```

输出：B析构
A析构

通过基类A指针撤销一个派生类B对象时，如果~A为virtual，调用B,A类中的析构函数

6.1 虚函数

建议：

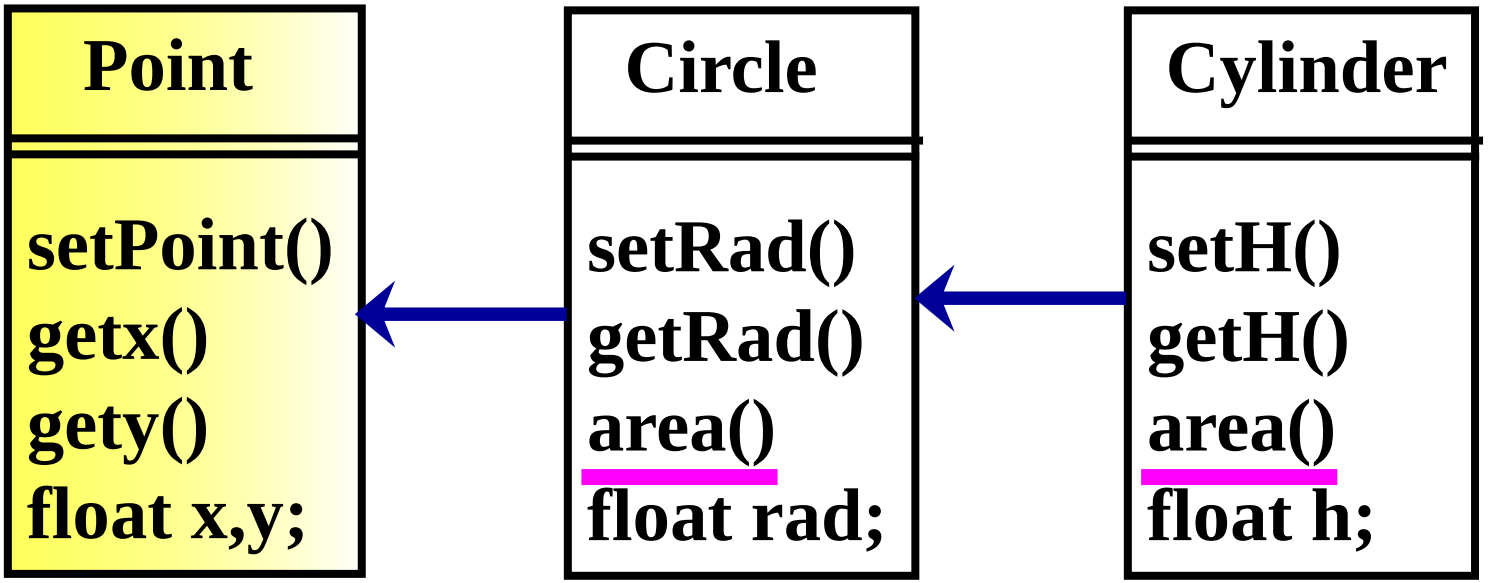
对每个析构函数都说明为virtual,增加通用性

注意：

- **构造函数不能是虚函数：在对象完全构造之前运行。**

6.2 纯虚函数

例：



如何体现多态性？

6.2 纯虚函数

方法一：

可以在基类Point中加一个area函数，并声明为虚函数：

```
virtual float area( ) {  return 0;  }
```

返回值为0，表示“点”是没有面积的。

方法二：

可以在基类Point中加一个area函数，并声明为纯虚函数：

```
virtual float area( ) =0;
```

6.2 纯虚函数

概念：

- 被标明为不具体实现的虚拟成员函数。

适用场合：

- 定义一个基类时，无法定义基类中虚函数的具体实现，其实现依赖于不同的派生类。

定义格式：

定义纯虚函数的一般格式为：

virtual 返回类型 函数名（参数表）=0;

```
class  A{
    protected:  int x;
    public:
          A(){x =1000;}
          virtual void  print()=0;
};
class B:public A{
          int y;
     public:
          B(){ y=2000;}
           void  print(){cout <<“y=”<<y<<‘\n’;}
};
class C:public A{
          int z;
     public:
          C(){z=3000;}
          void  print(){cout <<“z=”<<z<<‘\n’;}
};
```

定义纯虚函数

抽象类

重新定义纯虚函数

重新定义纯虚函数

```
int  main()
{   A  *pa;
    B  b;
    C  c;
     pa=&b;
    pa->print();
    pa=&c;
    pa->print();
     A  a;
    pa=&a;
    pa->print( );
     ret
}
```

Error!抽象类不能定义对象

6.2 纯虚函数

说明：

含有纯虚函数的基类（抽象类）是不能用来定义对象的：纯虚函数没有实现部分(函数体);

“=0”并不表示函数返回值为0，只起形式上的作用，标识纯虚函数;

作为声明语句，最后应有分号。

纯虚函数只有函数的名字而不具备函数的功能，不能被调用。 它只是通知编译系统：“在这里声明一个虚函数，留待派生类中定义”。

信息科学与工程学院
王文博 2026

抽象类

把至少包含一个纯虚函数的类，称为抽象类。这种类只能作为派生类的基类，不能用来说明这种类的对象

可以定义指向抽象类的指针，即指向这种基类的指针。当用这种基类指针指向其派生类的对象时，必须在派生类中重载纯虚函数，否则会产生程序的运行错误

抽象类

作用

抽象类为抽象和设计的目的而声明，将有关的数据和行为组织在一个继承层次结构中，保证派生类具有要求的行为。

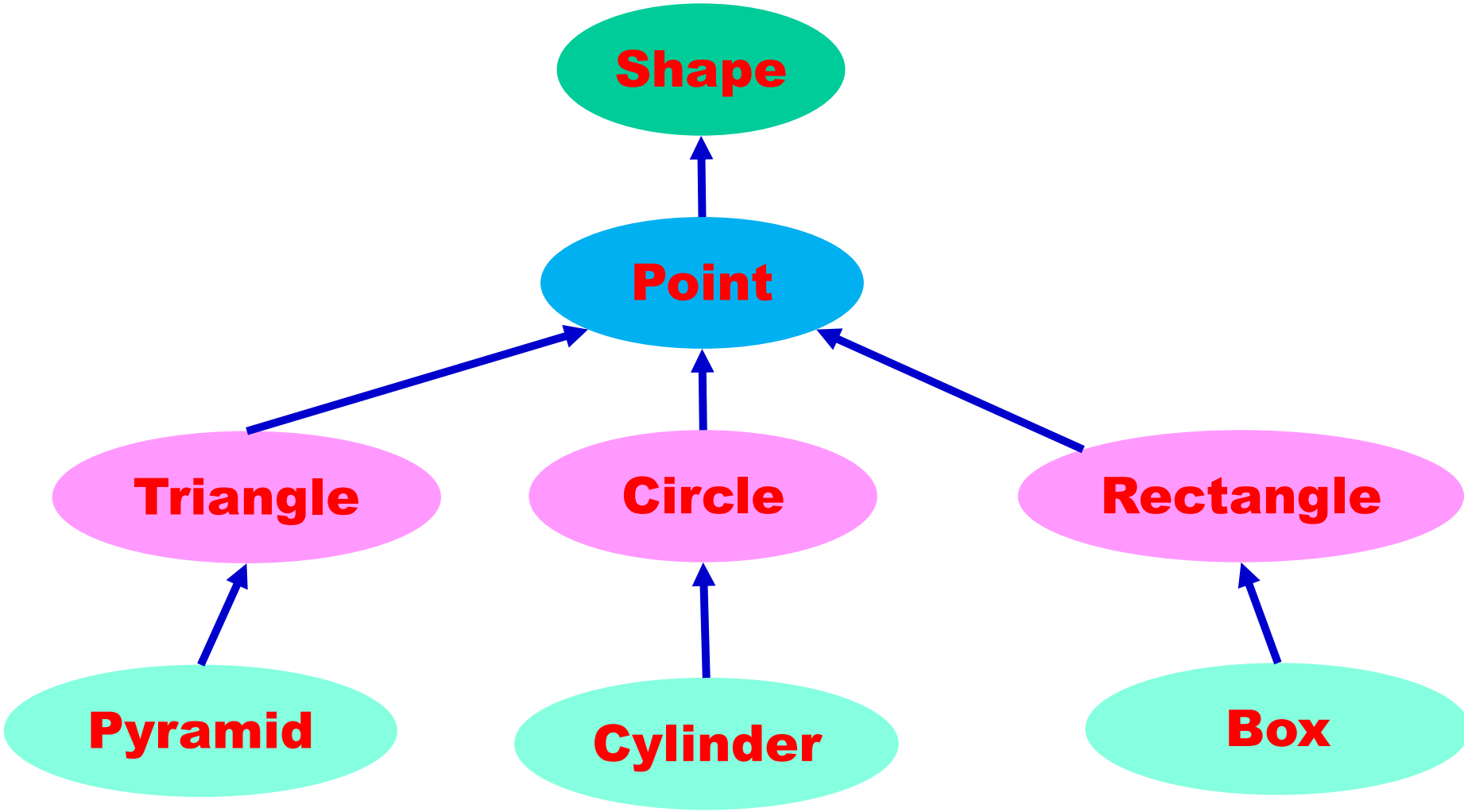
许多好的面向对象的系统，其层次结构的顶部是一个抽象类，甚至顶部有好几层都是抽象类。

如果在抽象类所派生出的新类中对基类的所有纯虚函数进行了定义，那么这些函数就被赋予了功能，可以被调用。这个派生类就不是抽象类，而是可以用来定义对象的具体类。

如果在派生类中没有对所有纯虚函数进行定义，则此派生类仍然是抽象类，不能用来定义对象。

信息科学与工程学院
王文博 2026

抽象类与虚函数的应用示例1



抽象类与虚函数的应用示例1

```
#include <iostream>
using namespace std;
class Shape
{
        public:
         virtual float area() {return 0.0;}
         virtual float volume() {return 0.0;}
         virtual void show()=0;
         virtual void shapeName () =0;
};
```

虚函数

纯虚函数

抽象类与虚函数的应用示例1

```
class Point:public Shape
{
  public:
    Point(float a=0,float b=0) {x=a;y=b;}
    void setPoint(float,float) {x=a;y=b;}
    float getX() const {return x;}
    float getY() const {return y;}
    void shapeName() {cout<<"Point:";}
    void show(){cout<<"["<<x<<","<<y<<"]"<<endl;}
  protected:
    float x,y;
}
```

纯虚函数

抽象类与虚函数的应用示例1

```
class Circle:public Point
{public:
  Circle(float x=0,float y=0,float r=0);
  void setRadius(float);
  float getRadius( ) const;
  float area( );
  void shapeName( ) {cout<<"Circle:";}
  void show()   {cout<<"圆心"; Point::show();
        cout <<"半径" <<radius<<endl;}
protected:
  float radius;
};
```

纯虚函数

抽象类与虚函数的应用示例1

```
Circle::Circle(float a,float b,float r):
Point(a,b),radius(r){ }
void Circle::setRadius(float r):radius(r){ }
float Circle::getRadius() const {return radius;}
float Circle::area() {return 3.14159*radius*radius;}
```

抽象类与虚函数的应用示例1

```
class Cylinder:public Circle
{public:
  Cylinder (float x=0,float y=0,float r=0,float h=0);
  void setHeight(float);
  float area();
  float volume();
  void shapeName() {cout<<"Cylinder:";}
  void  show(){  Circle::show();
          cout<<"高度:"<<height<<endl;}
protected:
  float height;
};
```

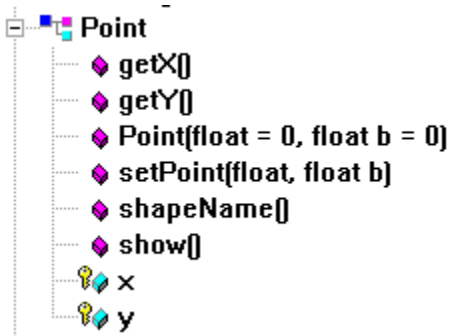

信息科学与工程学院
王文博 2026

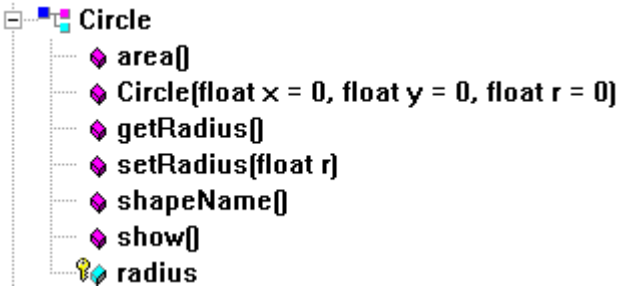
抽象类与虚函数的应用示例1

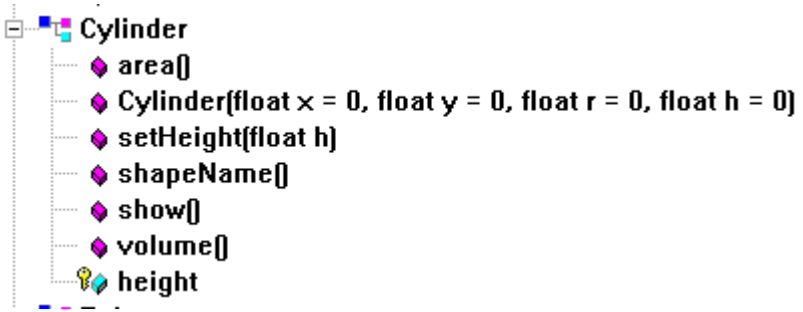
```
Cylinder::Cylinder(float a,float b,float r,float h) :
Circle(a,b,r),height(h) { }
void Cylinder::setHeight(float h) {height=h;}
float Cylinder::area()
{ return 2*Circle::area()+2*3.14159*radius*height;}
float Cylinder::volume()
{return Circle::area()*height;}
```

抽象类与虚函数的应用示例1

```
int main(){
    Point point(3.2,4.5)
    Circle circle(2.4,1.2,5.6);
    Cylinder cylinder(3.5,6.4,5.2,10.5);
    point.shapeName();
    point.show();
    circle.shapeName ();
    circle.show();
    cylinder.shapeName ();
    cylinder.show();
```









通过派生类对象调用函数

与工程学院
文博 2026

抽象类与虚函数的应用示例

定义基类的指针

通过基类指针实现多态

```
Shape *pt;
pt=&point;
pt->shapeName ();
pt->show();
cout<<"area"<<pt->area()<<"\tvolume:" <<pt->volume()<<endl;
pt=&circle;
pt->shapeName ();
pt->show();
cout<<"area"<<pt->area()<<"\tvolume:" <<pt->volume()<<endl;
pt=&cylinder;
pt->shapeName ();
pt->show();
cout<<"area"<<pt->area()<<"\tvolume:" <<pt->volume()<<endl;
        return 0;
}
```

抽象类与虚函数的应用示例 2

用虚函数来实现辛普生法求函数的定积分。在辛普生法中是用抛物线来代替，得出的公式为（区间必须为偶数**n**个相等区间）

$$\int_a^b f(x)dx \approx \frac{1}{3}\Delta x[y_0 + y_n + 4(y_1 + y_3 + \ldots + y_{n-1}) + 2(y_2 + y_4 + \ldots + y_{n-2})]$$

抽象类与虚函数的应用示例2

```
#include<cmath>
using namespace std;
class Simpson{
        double Intevalue,a,b; public:
        virtual double fun(double x)=0;
        Simpson(double ra=0,double rb=0)
      {  a=ra;          b=rb;          Intevalue=0;    }
        void Integrate(){
          double dx;
           int i;
          dx=(b-a)/2000;
          Intevalue=fun(a)+fun(b);
          for(i=1;i<2000;i+=2)Intevalue+=4*fun(a+dx*i);
          for(i=2;i<2000;i+=2)Intevalue+=2*fun(a+dx*i);
          Intevalue*=dx/3;
        }
        void Print(){cout<<"积分值="<<Intevalue<<endl;}
};
```

fun函数为纯虚函数

抽象类与虚函数的应用示例 2

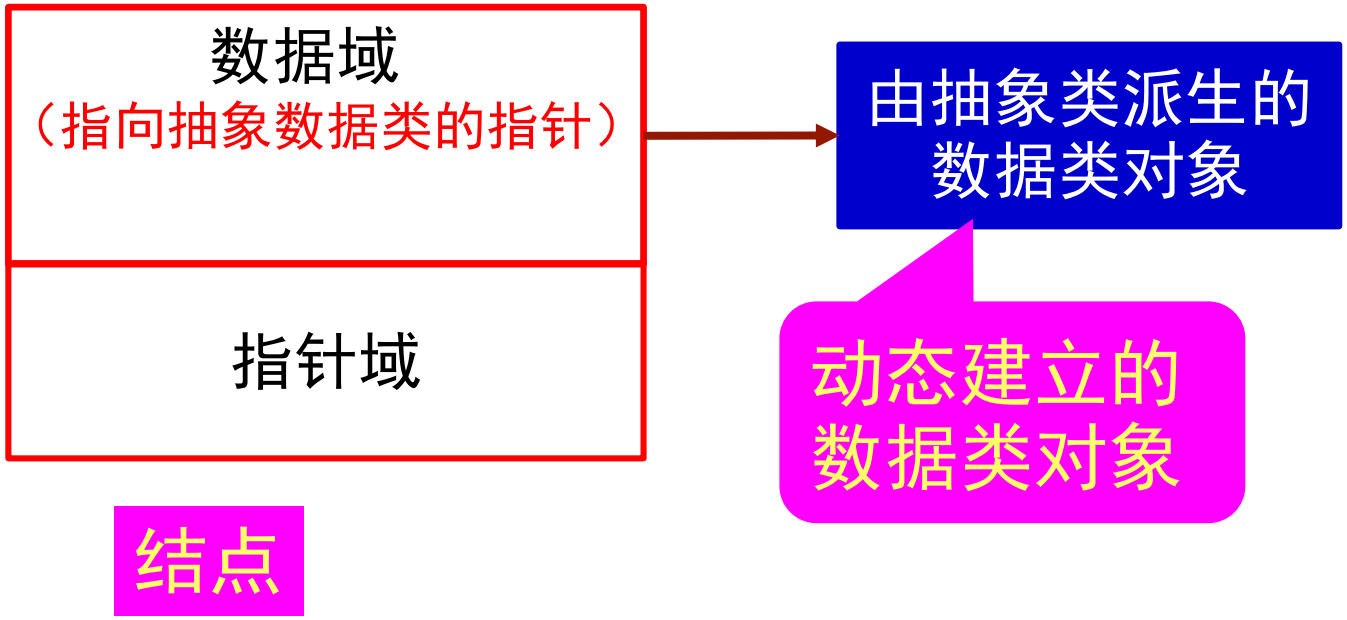
```
class A:public Simpson
{
public:
        A(double ra,double rb):Simpson(ra,rb){};
      double fun(double x){return sin(x);}
};
class B:public Simpson
{ public:
        B(double ra,double rb):Simpson(ra,rb){};
      double fun(double x){return exp(x);}
};
```

抽象类与虚函数的应用示例2

```
int main()
{
        A a1(0.0,3.1415926535/2.0);
        Simpson *s=&a1;
        s->Integrate();
        B b1(0.0,1.0);
        b1.Integrate();
        s->Print();
        b1.Print();
        return 0;
}
```

继承与多态的应用-单链表派生类

不用模板类，采用虚函数实现多态性，达到通用的目的。



```
class Object{
        public:
                Object(){}
                virtual bool operator >(Object &)=0;
                virtual bool operator !=(Object &)=0;
                virtual void Print()=0;
                virtual ~Object(){}
};
```

```
class Node
{
        Object *info;
        Node *link;
        public:
                Node();
                ~Node();
                void InserAfter(Node *P);
                Node *RemoveAfter();
                void Linkinfo(Object *obj);
        friend class  List;
};
```

数据域用指针指向数据类对象

```
Node * Node::RemoveAfter()
{
        Node *tempP=link;
        if(link=NULL) tempP=NULL;
        else link=tempP->link;
        return tempP;
}
```

```
Node::Node()
{
   info=NULL;
   link=NULL;
}
Node::~Node()
{
   cout<<"删除结点"<<'\t';
   if(info) delete info;
}
```

```
void Node::Linkinfo(Object *obj) {info=obj;}
```

```
void Node::InserAfter(Node *p)
{       p->link=link;
        link=p;
}
```

```
class List
{
    Node * head, * tail;          //链表头指针和尾指针
public:
    List();                       //构造函数，生成头结点(空链表)
   ~List();                       //析构函数
   void MakeEmpty();              //清空链表，只余表头结点
   Node * Find(Object &obj);      //搜索结点，返回地址
   int Length();                  //计算单链表长度
   void PrintList();              //打印链表的数据域
   void InsertFront(Node* p);     //可用来向前生成链表
   void InsertRear(Node* p);      //可用来向后生成链表
   void InsertOrder(Node* p);     //按升序生成链表
   Node* CreatNode();             //创建结点(孤立结点)
   Node* DeleteNode(Node* p);     //删除指定结点
};
```

```
void List::PrintList()
{
    Node* tempP = head->link;
     while (tempP != NULL)
           {
            tempP->info->Print();
            tempP = tempP->link;
           }
     cout << endl;
}
```

```
Node* List::Find(Object &obj)
{
    Node* tempP = head->link;
    while (tempP != NULL &&                    != obj)
           tempP = tempP->link;
    return tempP;
}
```


信息科学与工程学院
王文博 2026

```
void List::InsertOrder(Node* p)
{
    Node* tempP = head->link, * tempQ = head;
    while (tempP != NULL) {
        if (                                    ) break;
        tempQ = tempP;
        tempP = tempP->link;
    }
    tempQ->InserAfter(p);
    if (tail == tempQ) tail = tempQ->link;
}
```


信息科学与工程学院
王文博 2026

```
class StringObject: public Object
{       string sptr;
  public:
        StringObject() {sptr="";}
        StringObject(string s) {sptr=s;}
       ~StringObject(){ cout<<“数据类析构”<<endl;}
        bool operator>(Object &);
        bool operator!=(Object &);
        void Print();
};
```

由抽象类派生的数据类

```
class ComplexObject: public Object
{
        float imag,real;
  public:
        ComplexObject() {real=imag=0;}
        ComplexObject(float r, float i) {real=r,imag=i;}
         bool operator>(Object &);
        bool operator!=(Object &);
        void Print();
};
```

继承与多态的应用-单链表派生类

```
bool StringObject::operator>(Object & obj)
{

   StringObject & temp=(StringObject &)obj;
    return sptr>temp.sptr;
}
```

参数是基类的引用

```
bool StringObject::operator !=(Object & obj)
{
        StringObject & temp=(StringObject &)obj;
        return sptr!=temp.sptr;
}
```

```
void StringObject::Print(){cout<<sptr<<'\t';}
```

继承与多态的应用-单链表派生类

```
bool ComplexObject::operator>(Object & obj
{
   Complex Object & temp=(ComplexObject &)obj;
   float m1=real*real+imag*imag;
   float m2= temp.real*temp.real+temp.imag*temp.imag
 return m1>m2;
}
```

参数是基类的引用

```
bool Complex Object::operator !=(Object & obj)
{
          Complex Object & temp=(Complex Object &)obj;
         return real!=temp.real ||imag!=temp.imag;
}
```

```
void ComplexObject::Print()
{cout<<'('<<real<<','<<imag<<")\t";}
```

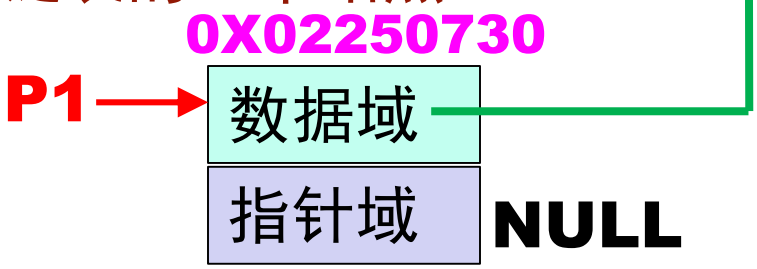
```
int main()
{
    Node * P1;
    StringObject* p;
    List list1,list2,list3;
    char *a[3]={"dog","cat","bear"};
    for(int i=0;i<3;i++){
        p=new StringObject(a[i]);
        P1=list1.CreatNode();
        P1->Linkinfo(p);
        list1.InsertFront(P1);
        p=new StringObject(a[i]);
        P1=list2.CreatNode();
        P1->Linkinfo(p);
        list2.InsertRear(P1);
     }
    list1.PrintList();
    list2.PrintList();
    p=new StringObject(a[2]);
    P1=list1.Find(*p);
    delete p;
    if(P1!=NULL)
    {
        cout<<"删除cat"<<endl;
        P1=list1.DeleteNode(P1);
        delete P1;
        list1.PrintList();
    }
    else cout<<"未找到"<<endl;
    list1.MakeEmpty();
    list1.PrintList();
    for(int i=0;i<3;i++){
      p=new StringObject(a[i]);
      P1=list3.CreatNode();
     P1->Linkinfo(p);
     list3.InsertOrder(P1);
    }
    list3.PrintList();
    cout<<"程序结束"<<endl;
    return 0;
}
```

```
p=new StringObject(a[i]);
P1=list1.CreatNode();
P1->Linkinfo(p);
list1.InsertFront(P1);
```

- 创建**String Object** 动态无名对象

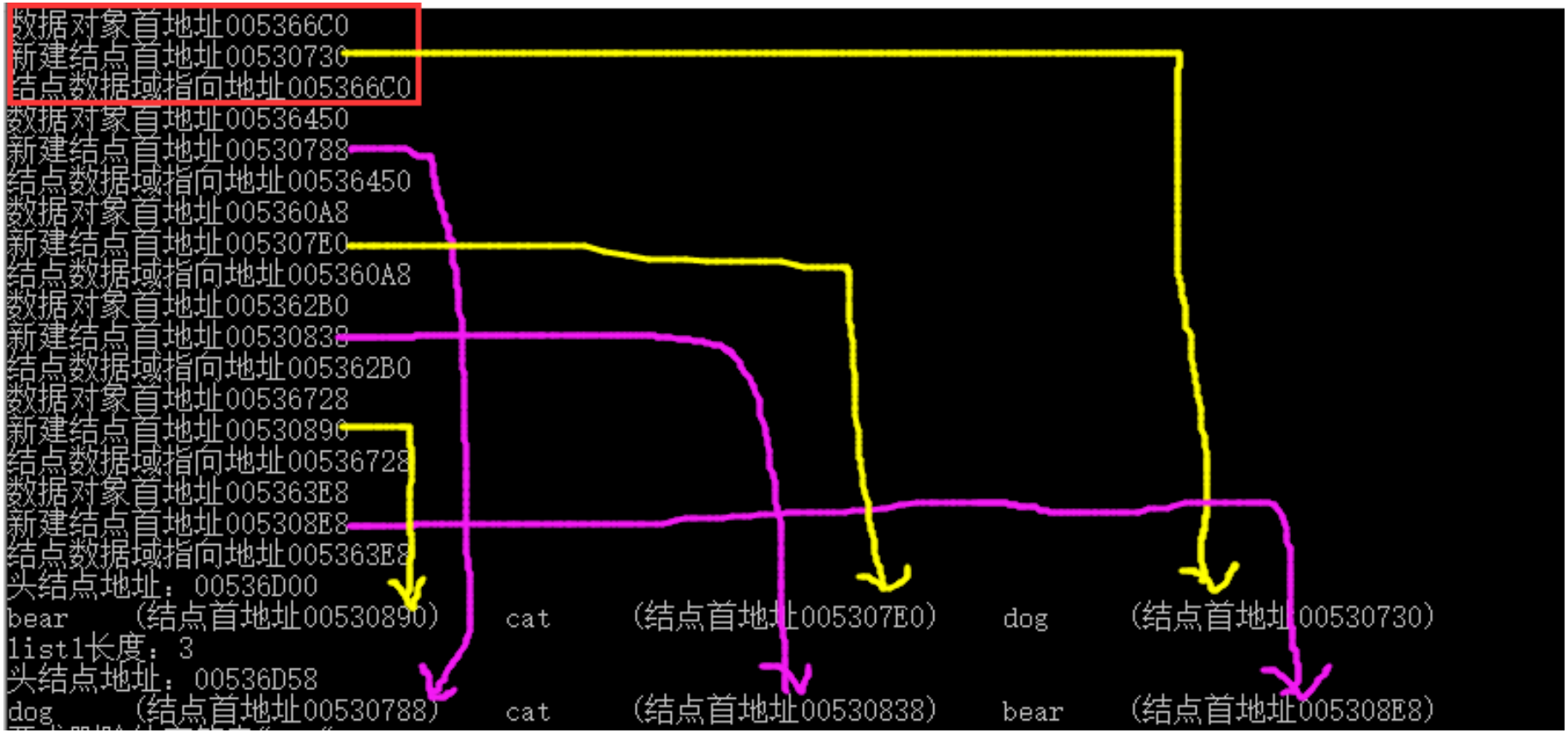


- 创建链表的一个结点



- 将结点的数据域指针指向创建的**String Object** 动态无名对象

- 将结点前向法插入链表中


数据对象首地址005366C0
新建结点首地址00530730
结点数据域指向地址005366C0
数据对象首地址00536450
新建结点首地址00530788
结点数据域指向地址00536450
数据对象首地址005360A8
新建结点首地址005307E0
结点数据域指向地址005360A8
数据对象首地址005362B0
新建结点首地址00530838
结点数据域指向地址005362B0
数据对象首地址00536728
新建结点首地址00530890
结点数据域指向地址00536728
数据对象首地址005363E8
新建结点首地址005308E8
结点数据域指向地址005363E8
头结点地址：00536D00
bear (结点首地址00530890) cat (结点首地址005307E0) dog (结点首地址00530730)
list1长度：3
头结点地址：00536D58
dog (结点首地址00530788) cat (结点首地址00530838) bear (结点首地址005308E8)

代码示例

```
p=new StringObject(a[2]);
 P1=list1.Find(*p);
delete p;
```

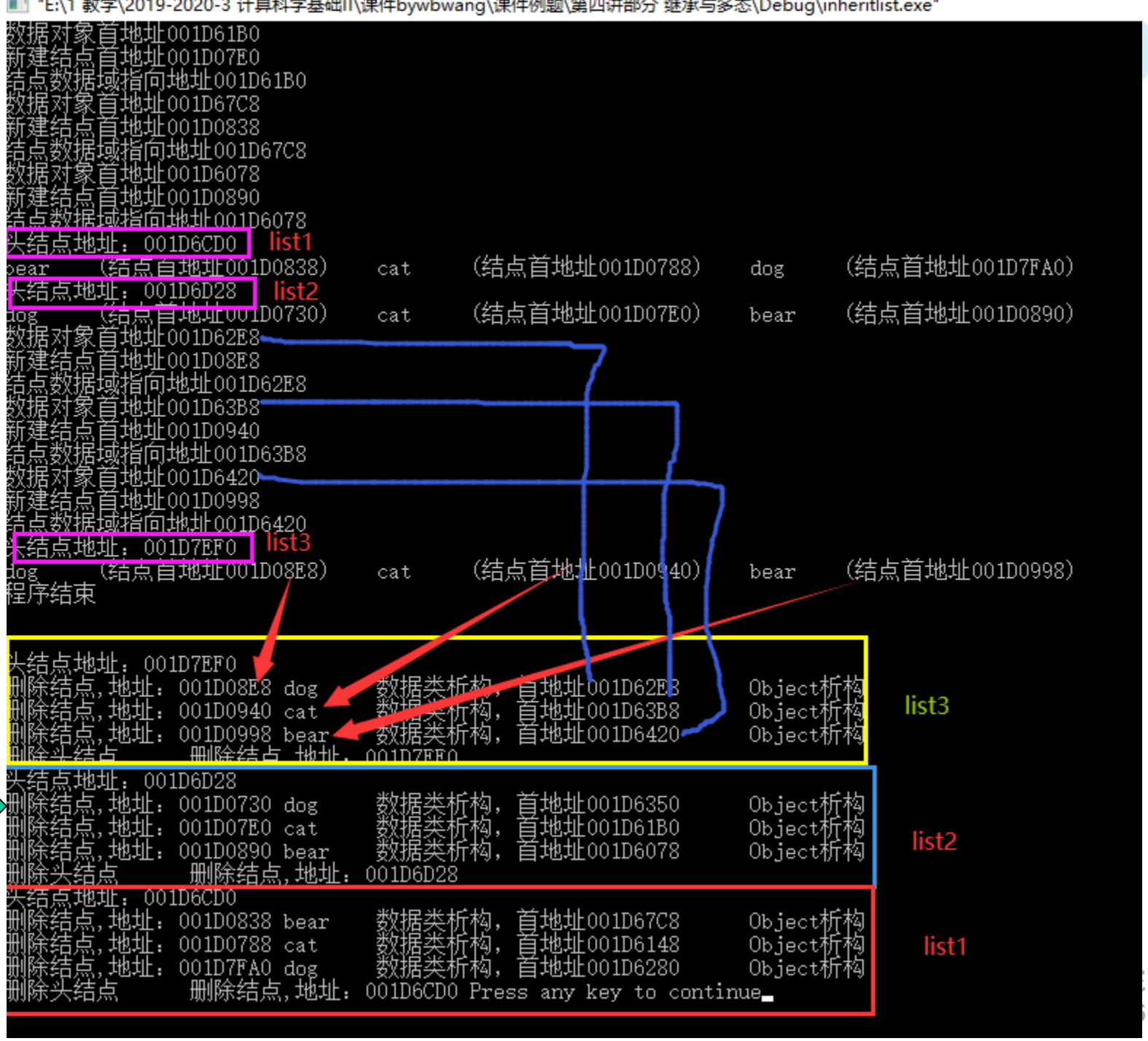
- 调用派生类**String Object** 析构函数
- 调用 基类抽象类**Object** 析构函数

```
P1=list1.DeleteNode(P1);
delete P1;
```

- 调用**Node**析构函数
- 调用**String Object**析构函数
- 调用**Object**析构函数

注意：结点的数据域**info**指向一个动态对象

程序结束析构过程


"E:\1 教学\2019-2020-3 计算科学基础II\课件bywbwang\课件例题\第四讲部分 继承与多态\Debug\inheritlist.exe"
数据对象首地址001D61B0
新建结点首地址001D07E0
结点数据域指向地址001D61B0
数据对象首地址001D67C8
新建结点首地址001D0838
结点数据域指向地址001D67C8
数据对象首地址001D6078
新建结点首地址001D0890
结点数据域指向地址001D6078
头结点地址：001D6CD0
list1
bear (结点首地址001D0838) cat (结点首地址001D0788) dog (结点首地址001D7FA0)
头结点地址：001D6D28
list2
dog (结点首地址001D0730) cat (结点首地址001D07E0) bear (结点首地址001D0890)
数据对象首地址001D62E8
新建结点首地址001D08E8
结点数据域指向地址001D62E8
数据对象首地址001D63B8
新建结点首地址001D0940
结点数据域指向地址001D63B8
数据对象首地址001D6420
新建结点首地址001D0998
结点数据域指向地址001D6420
头结点地址：001D7EF0
list3
dog (结点首地址001D08E8) cat (结点首地址001D0940) bear (结点首地址001D0998)
程序结束
头结点地址：001D7EF0
删除结点,地址：001D08E8 dog 数据类析构，首地址001D62E8 Object析构
删除结点,地址：001D0940 cat 数据类析构，首地址001D63B8 Object析构
删除结点,地址：001D0998 bear 数据类析构，首地址001D6420 Object析构
删除头结点 删除结点,地址：001D7EF0
list3
头结点地址：001D6D28
删除结点,地址：001D0730 dog 数据类析构，首地址001D6350 Object析构
删除结点,地址：001D07E0 cat 数据类析构，首地址001D61B0 Object析构
删除结点,地址：001D0890 bear 数据类析构，首地址001D6078 Object析构
删除头结点 删除结点,地址：001D6D28
list2
头结点地址：001D6CD0
删除结点,地址：001D0838 bear 数据类析构，首地址001D67C8 Object析构
删除结点,地址：001D0788 cat 数据类析构，首地址001D6148 Object析构
删除结点,地址：001D7FA0 dog 数据类析构，首地址001D6280 Object析构
删除头结点 删除结点,地址：001D6CD0 Press any key to continue
list1